

The main objective of this project was to develop a command-line AI programming assistant that acted as a fully autonomous agent instead of a chatbot. In order to accomplish this, the system would receive a type of function in natural language form and determine what should occur by combining responsibility (for its operations) with the selection of tools via the MCP servers (operations performed). For its operations, the system would select a tool or multiple tools based on its understanding of the function and then execute the selected tool(s) after which the system would observe the results of tool execution. The system would repeat these steps of iteration until the function was completed or until it was unable to complete it (i.e., it got stuck).

Modularity was a very important consideration throughout the project, and as a result, we organized the project and created varying system components (layers) that operate independently of one another: the Command-Line Interface (CLI), the Agent Loop, the Provider Abstraction, MCP Integration, Approval/Execution Flow, and the Custom (or Automated) Retrieval of All Generated Data (RAG) Server. This project also meets the project's requirement that the AI programming assistant be perceived by users as a true engineering tool (and not merely as a prototype) by creating separate and distinct layers between the Command-Line Interface (CLI), the Agent Loop, the Provider Abstraction, MCP Integration, an approval/execution flow, and the Custom Retrieval of Generated Data (RAG) Server.

The CLI is responsible for the User Experience (e.g., prompting users, rendering streamed output, displaying tool calls, and asking users for approval prior to executing anything). The Agent Loop determines the steps necessary to accomplish a function. When separating the User Experience (UX) Logic from the Reasoning Logic, it created a cleaner code base and made debugging easier by allowing developers to work on UX issues and Reasoning issues separately, without interfering with one another. Additionally, separating the UX from the Model Logic enables the CLI to display the tools that were used and clearly indicates which tools were requested and approved by users, thus meeting the requirement to eliminate any UI-related issues from the project. The principal second design choice was designing an agent loop as a state machine rather than a loose while-loop with ad hoc conditionals. The planners progressed on to LangGraph (graphical user interface/web pages) style design with each node in the graph representing either a particular type of input handler (e.g. text-based interactions), mode classifier, tool executor, planner, parallel executor, synthesiser or final response node. It made it easier for people to reason about the system because every node has defined roles, which allows for clear transitions from one node to another. It supports both a debug mode that operates like a ReAct-type loop and a build mode that consists of planning and executing a task based on those plans. This strengthened the architecture's identity and enabled it to support both individual debugging tasks on a step-by-step basis and higher-level multi-step construction tasks.

The third significant design choice was the addition of support for a provider-agnostic architecture at the model layer. The assistant architecture now allows you to change which model (provider) is being used through configuration instead of having to hard code the assistant's connection to a specific provider. The plan also moved toward the LiteLLM architecture being the unifying interface between the planning model and the execution model,

with the planning node being a stronger model for reasoning/planning than the execution node, which may be a cheaper or faster model suited for executing/performing tasks such as the tool responsible for the task performed. Real-world development often sees different models used in the development workflow for different parts of the workflow, with stronger models better suited to planning and determining when to call a tool and much smaller, local models are often sufficient for less-expensive, structured subtasks. It also makes the assistant easier to explain because changing providers would be a configuration change rather than a code change.

The design also strongly emphasised tool integration based on MCP. The assistant now connects to multiple MCP servers to find tools for inclusion into the assistant and exposes them through a common tool registry. There are three types of tool categories that must be supported: filesystem servers allow local interaction with your codebase; external resource servers extend the assistant's reach into areas of your project beyond the local area; and a custom RAG server

The dynamic tool loading feature provided the model with context based on the documentation that it was unlikely to retain on its own. Dynamic tool loading made the assistant appear to be a more general-purpose coding tool rather than an application or program built upon a preset of limited capabilities. We made a conscious design choice to handle approval and safety differently than we had previously done with regard to tools used. Instead of using all tools in the same manner, we categorized tools into categories based on whether they performed a Read Action or a Write Action. Read Actions are allowed to run automatically, but Write Actions will require varying degrees of confirmation based on the mode in which they were initiated. Debug Mode will require that each Write Action gets confirmation before execution. Therefore, Debug Mode is safer to use and easier to audit during the development of an application. In Build Mode, we request confirmation for each Write Tool (Tool Type) as a batch of Write Action confirmations. This method allows the system to maintain a smooth operational flow while still providing users with control over tool use.

By using these methods, we have achieved a good balance between providing the user with a level of autonomy while also ensuring user safety. The assistant's behavior maintains the appearance of being agentic, yet not reckless or opaque.

The custom RAG MCP server was the most interesting technical subsystem of this project. We developed a system that ingests, separates, embeds and stores documentation in a permanent Chroma database that can be accessed from future sessions. This approach provides both practical benefits as well as valuable user experience gains. Without the ability to retain saved indices, the usable speed of the tool would decrease and it would be more challenging to demonstrate functionality. Storing documentation permanently allows the system to take on more of a "Digital Assistant" feel. We kept RAG server as its own entity within the repository, as this allowed for greater simplicity in explaining how the architecture works and how to test the subsystem independently of the main agent loop.

The advanced RAG acquisition technique used in this project was Fusion Retrieval. Fusion Retrieval is more sophisticated than a single-query naive approach because it takes an original question and creates up to multiple related queries to search. For each query, Fusion Retrieval retrieves and deduplicates results to return chunks found in one or more duplicate results, then

ranks by frequency of occurrence in the returned results to provide the best answer for that question in comparison with the original question. This is significant for documentation search, where users may phrase the question in a way that does not match the exact wording in the documents. A single-query retriever could miss relevant chunks of information, causing the user to receive an incorrect or incomplete answer. Fusion Retrieval allows for improved recall (what is found) without being overly complex to implement, which allowed us to use it as the basis of a class project: it is clearly more advanced than the baseline retrieval techniques, but still simple enough to explain during a reflection and to show in the live demo.

In a coding assistant, the three essential elements of coverage, robustness, and explainability are all dependent on each other. The ability to retrieve the correct chunk of documentation will ultimately determine if the model is able to use the library properly, pick the correct API pattern to use, and create an implementation plan that matches the documentation. The expansion of the search by using multiple rewrites of the question allows for a higher probability that relevant information will be located and a lower probability that the model will revert to using an ambiguous prior knowledge or hallucinate about the use of the library's functions. The development of the improved RAG method did not only provide improved retrieval quality; it provided improved reasoning because the advanced retrieval process provided the model with improved evidence to work from. As a result, the RAG subsystem became more than just a checklist item; rather, it became a significant contributor to the overall capability of the assistant.

In making comparisons of the LLM, the architecture would naturally lend itself to comparing the more capable agent or planner-based model against the less expensive executor model on the same task of coding. A logical point of comparison would be between a cloud-based reasoning service (e.g., Anthropic or OpenAI) and a locally run Ollama model functioning in the role of executor. On the same task, the stronger model accounts for planning, determining when to call tools, interpreting outputs from tools, and making corrections for ambiguity; conversely, the smaller local model is less expensive and faster to run but has less reliability for performing long-term reasoning and making decisions based on ambiguous intermediate outcomes. For this reason, the architecture distinguishes between the `agent_model` and `executor_model`: the basic idea is that both types of models bring their own sets of strengths to the particular stages of the system. From this comparison, we can see that strong reasoning is most important at the time of making decisions, while the less expensive models are sufficient for narrow, well-defined tasks. Requesting a model to summarize the next step after reading a short set of results differs greatly from a model having to control an entire loop of autonomous coding. This insight had a significantly greater impact on the development of the system than raw benchmark metrics provided. Rather than developing a system that required everything to go through either one expensive model or one lower-quality local model, we focused on designing the overall architecture around specialization. As a result, we achieved a much more realistic balance between speed, quality, and cost.

Another conceptual challenge was balancing autonomy with control. Although it may be nice to have the convenience of a completely autonomous assistant, in reality, users need to see what the system is doing (especially given that files may be edited and commands may be executed), and that is why we have placed great importance on visibly calling tools, streaming progress

updates, and prompting for user confirmation. By providing users with these functionalities, the assistant has become much more trustworthy and is much easier to evaluate. Additionally, users were able to see the assistant in action, rather than only seeing the end result. Though the project has primarily been focused on functionality, the terminal UX is also very important because it determines whether or not the assistant can be considered understandable or an opaque black box.

There are many additional areas where we would focus our efforts if we had a week of additional time to invest in improving the system. First, we would enhance dependency-aware plan execution in build mode (in the MVP, the assistant is limited to expanding tasks as though they all occurred independently of one another). The next generation of the assistant would keep track of the real dependencies that exist between steps and schedule those dependencies in a more intelligent way. Second, we would develop better evaluation and instrumentation by logging how many tokens the model consumes, how long it takes for the model to execute, and what types of errors occur (this would improve the quality of both debugging and the final reflection). Third, we would make modifications to improve edit safety through a combination of diff previews, rollback capabilities, and undo functionality. All of these enhancements would increase the assistant's utility as a tool for real-world development scenarios. If we had additional time, we would continue to improve the RAG server, including using semantic chunking to improve access to documentation, building a better pipeline for re-ranking results, and adding support for multiple documentation libraries rather than limiting the assistant's use to a single library.

In conclusion, our experience building the AI coding assistant reinforced our understanding that simply connecting an LLM to a user's terminal will not lead to an effective assistant. The most challenging aspect of developing such a system will always be the orchestration of the various components: determining what is next in terms of actions to execute, managing the safe use of the tools involved, ensuring that the assistant is grounded in the appropriate documentation, and making the process visible to the user throughout the process. The most significant benefit of the modular architecture of our solution is that we have separated these concerns into manageable components: the agent loop for reasoning, the provider layer for abstracting model access, the MCP for supporting the extension of tools, and the RAG server for supporting access to documentation. Separation of concerns within the assistant allows for easier iteration and testing of progress, easier explanation of what went into the design, and offers the opportunity to add new capabilities in the future.